

Technical and Demo

WoundScope — Technical Walkthrough & Demo Script

Written in plain language. Use this to answer judges' "how did you actually do it?" questions, and to run the 2.5-minute demo.

Part 1 — The technical problems and how we solved them

Each section is: **the problem** → **what we did** → **why it works**. No jargon.

1. The data source fails 30% of the time (rate limiting)

Problem. The hospital API randomly rejects about 1 in 3 requests (a “429 — too many requests”). With ~1,700 requests needed for 300 patients, naive code would lose a chunk of the data and never know.

What we did. - When a request is refused, we **wait and retry** — and we wait a little longer each time (this is called *exponential backoff*), and we respect the “try again in N seconds” hint the server sends back. - We run many requests **at the same time** instead of one-by-one, so the flakiness doesn't slow us to a crawl. - After the main pass, we run a **completeness check**: “does any patient have zero records?” If so, we **re-fetch just those** until everyone is complete.

Why it works. The failures are random, so retrying eventually succeeds. The backfill is the safety net. Result: **0 patients dropped** out of 300, even with ~530 refusals in a run. (*We literally caught a bug where 47 patients were being silently lost, and this is what fixed it.*)

One-liner for judges: “Retries with backoff for the random failures, plus a final backfill pass that guarantees no patient is left with missing data.”

2. Two different patient IDs for the same person

Problem. Insurance and diagnoses are looked up by a text ID like FA-001, but notes and assessments use a number like 1. Mix them up and you attach the wrong records to the wrong patient.

What we did. We pull the patient list first, build the FA-001 → 1 map, and only then fetch everything else using the correct ID for each endpoint.

3. The clinical notes are a mess — in several formats

Problem. The documentation showed one tidy note format. In the **real** data, that format **doesn't exist at all**. Instead there are **4 different note styles** (plus 2 assessment styles), and the label on each note doesn't tell you which style it is.

What we did. We figured out the 4 real styles by reading actual samples, and we **detect each one by its structure** (the shape of the text), then parse it with rules tuned to that style. We also handle the assessment “survey” format (question/answer pairs) — which is where most of the wound **measurements** actually live.

Why it works. Parsing each format on its own terms is far more accurate than one generic pattern. Fixing the assessment format alone recovered the wound **depth** for **285 of 300** patients — it had been almost entirely missed before.

One-liner: “We reverse-engineered the real note formats from the data and parse each by its structure — the docs’ example format wasn’t actually in the data.”

4. Some notes describe **TWO** wounds at once

Problem. A single note often documents two wounds (e.g., a foot ulcer *and* an ankle wound). For billing, **each wound is billed separately**, so missing the second one loses money.

What we did. We split the text per wound and then **group mentions of the same wound together** — even when the same wound appears in both a note and an assessment — by matching on body location or measurements. Different wounds stay separate.

Why it works. We never silently bill one and drop the other. The biller sees both, and the second wound isn’t lost.

5. The text is often garbled or abbreviated

Problem. Real notes contain things like Rightlowerle (no spaces) or shorthand like mod / serosang.

What we did. A repair layer fixes mangled locations (Rightlowerle → right lower leg) and expands abbreviations to standard terms before we use them.

6. The note sometimes never says what kind of wound it is

Problem. One note style states measurements but **no wound type**.

What we did. When the type is missing from the note, we **fill it in from the patient’s diagnosis code** (the ICD-10 code, which reliably encodes the wound type). We also expanded our code coverage to catch all the real wound types — pressure, diabetic, venous, arterial, burns, surgical, abscess.

7. “How do we know the answer is trustworthy?”

Problem. A billing decision a human can’t verify is useless — and risky.

What we did. Three things: - **Confidence score** — how complete and how reliable the source was, per patient. - **Evidence** — we keep the **exact sentence** each value came from, shown in the UI. - **Provenance** — which record (diagnosis / note / assessment) each field came from.

Why it works. A biller can check any decision against the original words in seconds. Nothing is a black box — which is what makes them actually use it.

8. The actual billing decision

Problem. Turn all of the above into one safe call per patient.

What we did. Check the three Medicare Part B rules in order — active wound, active Part B coverage, complete documentation (size + drainage) — and route: - **Auto-accept** if a patient has at least **one fully-documented wound** (they’re billable on that wound). If they have other, messier wounds, we **say so** so those aren’t missed. -

Flag for review if eligible but the documentation is incomplete or ambiguous. - **Reject** if there's no wound, the wrong insurance, or the data is unreliable.

Why it works. It mirrors how a biller actually reasons, and it's **conservative where it matters** — it never auto-bills on weak data (that would risk an improper Medicare claim).

9. Privacy (HIPAA)

Problem. This is patient health data; mishandling it is a legal issue.

What we did. Patient names are **masked by default** (you reveal them only when needed), everything runs **locally** (no data sent to outside services), and **every decision is logged**. The demo uses **synthetic** patients, so no real data is involved.

10. Built by four people without stepping on each other

Problem. Four parallel versions; merging is usually painful.

What we did. Each person owned a piece, and we merged the strongest of each into one build: the resilient ingestion + note parsers, the multi-wound grouping + evidence, the full diagnosis-code coverage + text repair, and the routing + privacy + dashboard.

Part 2 — Likely judge questions (with simple answers)

“How did you handle the rate limiting?” Retry the refused requests with increasing wait times, run many in parallel so it stays fast, then a final backfill pass re-fetches anyone still incomplete. Zero patients dropped despite ~30% failures.

“How did you parse such messy notes?” We didn't trust the documentation's example — we read the real data, found 4 actual note formats, and parse each by its structure. The biggest win was parsing the assessment survey format, which recovered wound depth for 285 of 300 patients.

“How do you know your extraction is correct?” Every value keeps the exact source sentence (evidence) and which record it came from (provenance), plus a confidence score. You can verify any decision against the original note instantly.

“What if a patient has more than one wound?” We detect and separate them, and bill-accept on the documented one while flagging that others exist — so no wound is silently missed.

“What about wrong or missing data?” Missing measurements → flagged for a human, never guessed. Wrong insurance → rejected with the reason. Unreliable extraction → rejected rather than risk a bad claim.

“Is it compliant / safe with patient data?” PHI masked by default, processed locally, every decision audit-logged — HIPAA-aligned. Demo runs on synthetic data.

“What would you do next?” Add an AI assistant that writes the biller's summary (kept assistive, never decides), calibrate the confidence threshold on labeled data, and write decisions back into the platform's delivery format.

Part 3 — The 2.5-minute demo script

Read naturally while screen-sharing <http://localhost:8000>. Navigation in [brackets].

[0:00 — open the dashboard] “This is WoundScope. The second it loads, it’s already read **300 patients** from the hospital system and sorted them. Up top: **135 are clean and ready to bill, 10 need a human, 155 can’t be billed** — and only about half even have the right Medicare coverage. So instead of a biller reading 300 charts, they work the 10 in the middle.”

[0:30 — click Auto-Accept filter → open a clean patient] “Here’s one we marked ready to bill. This popup shows its work — the **evaluation log**: active wound? yes. Medicare Part B? yes. Measurements complete? yes. Drainage documented? yes. Every check passes.”

[1:00 — expand Evidence] “And here’s the part judges care about — **proof**. This shows the exact sentence from the original nurse’s note that each value came from. Nothing is guessed, nothing is a black box.”

[1:25 — open the multi-wound patient, FA-009] “This one’s the tricky case. The note describes **two** wounds — a diabetic foot ulcer *and* an ankle wound. A simple tool bills the first and quietly misses the second, which is lost revenue. We catch both and flag them to be billed separately.”

[1:55 — open a Reject; then toggle the  PHI button] “Rejects are honest too — this one’s an HMO, not Part B, so there’s nothing to bill, and it says exactly that. And for privacy, patient names are hidden by default — I can reveal them when needed. Everything runs locally on synthetic data.”

[2:20 — close] “So: messy records in, a trustworthy work list out — every decision explained and backed by the original evidence. That’s WoundScope.”

Behind the demo — the one technical point to land

If a judge asks “what was the hardest part?”: **the data, not the UI**. The API fails 30% of the time (we guarantee completeness), and the real notes came in formats the documentation didn’t even mention (we reverse-engineered and parse all of them). The dashboard is just how we make that trustworthy to a human.